```
import  numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error,r2_score
#load the dataset
california =fetch_california_housing(as_frame=True)
df=california.frame

#Extract feature(AveRooms) and target(MedHousevaal)
X=df[['AveRooms']].values
y=df[['MedHouseVal']].values.reshape(-1,1)

print("Dataset loaded successfully by Jithin_34")
print(f"Feature Matrix Shape(X):{X.shape}")
print(f"Target Vector Shape(y):{y.shape}\n")

scaler_X=StandardScaler()
scaler_y=StandardScaler()

X_scaled=scaler_X.fit_transform(X)
y_scaled=scaler_y.fit_transform(y)
m=len(y)
print(df.head)
print(df.describe)
# Add a column of ones to X to handle the intercept/bias term(x_0=1)
X_b = np.c_[np.ones((m,1)),X_scaled]

theta_normal_scaled =np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y_scaled)
y_pred_normal_scaled = X_b.dot(theta_normal_scaled)
y_pred_normal=scaler_y.inverse_transform(y_pred_normal_scaled)

def compute_cost(X,y,theta):
    m=len(y)
    predictions = X.dot(theta)
    cost=(1/2*m)*np.sum((predictions-y)**2)
    return cost
def gradient_descent(X,y,theta,learning_rate,iteration):
    m=len(y)
    cost_history=[]
    for i in range(iteration):
        gradients = (1/m)*X.T.dot(X.dot(theta)-y)
        theta=theta-learning_rate*gradients
        cost_history.append(compute_cost(X,y,theta))
    return theta,cost_history

learning_rate=0.01
iterations=1500
initial_theta=np.zeros((2,1))
theta_gd_scaled,cost_history = gradient_descent(X_b,y_scaled,initial_theta,learning_rate,iterations)
y_pred_gd_scaled=X_b.dot(theta_gd_scaled)
y_pred_gd=scaler_y.inverse_transform(y_pred_gd_scaled)

mse_normal=mean_squared_error(y,y_pred_normal)
r2_normal=r2_score(y,y_pred_normal)
mse_gd=mean_squared_error(y,y_pred_gd)
r2_gd=r2_score(y,y_pred_gd)
print("------Model Evaluation Results---")
print(f"Normal Equation:\n -Mean squared Error(MSE): {mse_normal:.4f}\n -R-squared(R^2):{r2_normal:4f}")
print(f"Gradient Descent :\n-Mean Squared Error(MSE):{mse_gd:.4f}\n-R--squared(R^2):{r2_gd:.4f}\n")
print(f"Final weight(scaled-Normal Equation:Intercept={theta_normal_scaled[0][0]:.4f},slope={theta_normal_scaled[1]
[0]:.4f}")
print(f"Final weights(scaled).Gradient Descent:Intercept={theta_gd_scaled[0][0]:.4f},slope={theta_gd_scaled [1][0]:.4f}")

sort_axis=np.argsort(X.flatten())
X_sorted=X[sort_axis]
y_pred_gd_sorted=y_pred_gd[sort_axis]
plt.figure(figsize=(10,6))
plt.scatter(X[:1000],y[:1000],color='gray',alpha=0.5,label='Actual Date points(subset)')
plt.plot(X_sorted,y_pred_gd_sorted,color='Crimson',linewidth=3,label='Fitted Regression Line')

plt.title('Linear Regression:Averooms vs Median House value',fontsize=14)
plt.xlabel('Average Rooms per Dwelling(AveRooms)',fontsize=12)
plt.ylabel('Median House value(in $100,000s)',fontsize=12)
plt.xlim(0,15)
plt.ylim(0,5.5)
```

```
plt.legend(fontsize=11)
plt.grid(True,linestyle=':',alpha=0.6)
plt.show()

plt.figure(figsize=(8,4))
plt.plot(range(iterations),cost_history,color='navy',linewidth=2)
plt.title('Gradient Descent cost convergence curve')
plt.xlabel('Iterations')
plt.ylabel('cost J(theta)')
plt.grid(True,linestyle=':')
plt.show()

===op

dataset Loaded successfully by Jithin_34.
Feature Matrix Shape(X):(20640, 1)
Target Vector Shape(y):(20640, 1)

<bound method NDFrame.head of        MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0      8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1      8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2      7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3      5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4      3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85
...       ...       ...       ...        ...         ...       ...       ...
20635  1.5603      25.0  5.045455   1.133333       845.0  2.560606     39.48
20636  2.5568      18.0  6.114035   1.315789       356.0  3.122807     39.49
20637  1.7000      17.0  5.205543   1.120092      1007.0  2.325635     39.43
20638  1.8672      18.0  5.329513   1.171920       741.0  2.123209     39.43
20639  2.3886      16.0  5.254717   1.162264      1387.0  2.616981     39.37

       Longitude  MedHouseVal
0        -122.23        4.526
1        -122.22        3.585
2        -122.24        3.521
3        -122.25        3.413
4        -122.25        3.422
...          ...          ...
20635    -121.09        0.781
20636    -121.21        0.771
20637    -121.22        0.923
20638    -121.32        0.847
20639    -121.24        0.894

[20640 rows x 9 columns]>
<bound method NDFrame.describe of        MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0      8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1      8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2      7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3      5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4      3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85
...       ...       ...       ...        ...         ...       ...       ...
20635  1.5603      25.0  5.045455   1.133333       845.0  2.560606     39.48
20636  2.5568      18.0  6.114035   1.315789       356.0  3.122807     39.49
20637  1.7000      17.0  5.205543   1.120092      1007.0  2.325635     39.43
20638  1.8672      18.0  5.329513   1.171920       741.0  2.123209     39.43
20639  2.3886      16.0  5.254717   1.162264      1387.0  2.616981     39.37

       Longitude  MedHouseVal
0        -122.23        4.526
1        -122.22        3.585
2        -122.24        3.521
3        -122.25        3.413
4        -122.25        3.422
...          ...          ...
20635    -121.09        0.781
20636    -121.21        0.771
20637    -121.22        0.923
20638    -121.32        0.847
20639    -121.24        0.894

[20640 rows x 9 columns]>

------Model Evaluation Results---
Normal Equation:
 -Mean squared Error(MSE): 1.3008
 -R-squared(R^2):0.023088
```

```
Gradient Descent :
-Mean Squared Error(MSE):1.3008
-R--squared(R^2):0.0231

Final weight(scaled-Normal Equation:Intercept=0.0000,slope=0.1519
Final weights(scaled).Gradient Descent:Intercept=0.0000,slope=0.1519
```

===e1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

column_names = [
    'mpg', 'cylinders', 'displacement', 'horsepower',
    'weight', 'acceleration', 'model_year', 'origin', 'car_name'
]

df = pd.read_csv(
    "jitin/auto-mpg.data",
    sep=r"\s+",
    names=column_names,
    na_values="?"
)
print("Dataset loaded successfully by Jithin_34")


df.dropna(inplace=True)

# Feature and Target
X = df[['displacement']]
y = df['mpg']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

linear_mse = mean_squared_error(y_test, y_pred_linear)
linear_r2 = r2_score(y_test, y_pred_linear)

print("========== Linear Regression ==========")
print(f"MSE : {linear_mse:.4f}")
print(f"R²  : {linear_r2:.4f}")

degrees = [2, 3]

results = []

for degree in degrees:
    poly = PolynomialFeatures(degree=degree)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    y_pred = model.predict(X_test_poly)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results.append([degree, mse, r2])
```

```
print("\n====== Polynomial Regression Results ======\n")
print("{:<10} {:<15} {:<15}".format("Degree", "MSE", "R²"))

for degree, mse, r2 in results:
    print("{:<10} {:<15.4f} {:<15.4f}".format(degree, mse, r2))

 # ------------------------------------------------------
# Visualization: Linear vs Degree 2 vs Degree 3
# ------------------------------------------------------

plt.figure(figsize=(10,6))

# Scatter plot (Actual Data)
plt.scatter(
    X['displacement'],
    y,
    color='blue',
    alpha=0.5,
    label='Actual Data'
)

# Smooth X values for plotting curves
X_plot = np.linspace(
    X['displacement'].min(),
    X['displacement'].max(),
    300
).reshape(-1,1)

# ---------------- Linear Regression ----------------
y_linear = linear_model.predict(X_plot)

plt.plot(
    X_plot,
    y_linear,
    color='black',
    linewidth=2,
    label='Linear Regression'
)

# ---------------- Polynomial Degree 2 ----------------
poly2 = PolynomialFeatures(degree=2)

X_train_poly2 = poly2.fit_transform(X_train)
X_plot_poly2 = poly2.transform(X_plot)

model2 = LinearRegression()
model2.fit(X_train_poly2, y_train)

y_poly2 = model2.predict(X_plot_poly2)

plt.plot(
    X_plot,
    y_poly2,
    color='red',
    linewidth=2,
    label='Polynomial Regression (Degree 2)'
)

# ---------------- Polynomial Degree 3 ----------------
poly3 = PolynomialFeatures(degree=3)

X_train_poly3 = poly3.fit_transform(X_train)
X_plot_poly3 = poly3.transform(X_plot)

model3 = LinearRegression()
model3.fit(X_train_poly3, y_train)

y_poly3 = model3.predict(X_plot_poly3)

plt.plot(
    X_plot,
    y_poly3,
    color='green',
    linewidth=2,
    label='Polynomial Regression (Degree 3)'
)
```

```
# ---------------- Graph Details ----------------
plt.title("Linear Regression vs Polynomial Regression")
plt.xlabel("Engine Displacement")
plt.ylabel("Miles Per Gallon (MPG)")
plt.legend(loc='best')
plt.grid(True)

plt.show()
```

```
===op

Dataset loaded successfully by Jithin_34

========== Linear Regression ==========
MSE : 21.2275
R²  : 0.5841

====== Polynomial Regression Results ======
Degree     MSE             R²
2          20.6491         0.5954
3          20.8179         0.5921
```

===e2

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score


# Load Dataset
data = load_breast_cancer()
print("Dataset loaded successfully by Jithin_34")

X = data.data
y = data.target


# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)


# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


# MLE Logistic Regression
# Very weak regularization approximates MLE
mle_model = LogisticRegression(
    penalty='l2',
    C=1e6,
    max_iter=5000
)

mle_model.fit(X_train, y_train)

y_pred_mle = mle_model.predict(X_test)


# MAP with L2 Prior
map_l2 = LogisticRegression(
    penalty='l2',
    C=1.0,
```

```
    max_iter=5000
)

map_l2.fit(X_train, y_train)

y_pred_l2 = map_l2.predict(X_test)


# MAP with L1 Prior
map_l1 = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    C=1.0,
    max_iter=5000
)

map_l1.fit(X_train, y_train)

y_pred_l1 = map_l1.predict(X_test)


# Evaluation Function
def evaluate(name, y_true, y_pred):
    print("\n", name)
    print("Accuracy  :", accuracy_score(y_true, y_pred))
    print("Precision :", precision_score(y_true, y_pred))
    print("Recall    :", recall_score(y_true, y_pred))
    print("F1 Score  :", f1_score(y_true, y_pred))


# Evaluation Results
evaluate(
    "MLE Logistic Regression",
    y_test,
    y_pred_mle
)

evaluate(
    "MAP Logistic Regression (L2)",
    y_test,
    y_pred_l2
)

evaluate(
    "MAP Logistic Regression (L1)",
    y_test,
    y_pred_l1
)


# Coefficient Comparison
coef_df = pd.DataFrame({
    "MLE": mle_model.coef_[0],
    "MAP_L2": map_l2.coef_[0],
    "MAP_L1": map_l1.coef_[0]
})

print("\nCoefficient Comparison")
print(coef_df.head(10))

===op

dataset Loaded successfully by Jithin_34.

 MLE Logistic Regression
Accuracy  : 0.9385964912280702
Precision : 0.9848484848484849
Recall    : 0.9154929577464789
F1 Score  : 0.948905109489051

 MAP Logistic Regression (L2)
Accuracy  : 0.9736842105263158
Precision : 0.9722222222222222
Recall    : 0.9859154929577465
F1 Score  : 0.9790209790209791

 MAP Logistic Regression (L1)
```

```
Accuracy  : 0.9736842105263158
Precision : 0.9857142857142858
Recall    : 0.971830985915493
F1 Score  : 0.9787234042553191

Coefficient Comparison
          MLE    MAP_L2    MAP_L1
0    7.950604 -0.431904  0.000000
1  -13.650884 -0.387326  0.000000
2   33.058171 -0.393432  0.000000
3    9.158939 -0.465210  0.000000
4    4.075019 -0.071667  0.000000
5  226.224637  0.540164  0.000000
6 -111.293002 -0.801458  0.000000
7 -219.330380 -1.119804 -2.401425
8   34.096399  0.236119  0.013335
9 -137.922100  0.075921  0.000000
```

===e3

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score


column_names = [
'Pregnancies',
'Glucose',
'BloodPressure',
'SkinThickness',
'Insulin',
'BMI',
'DiabetesPedigreeFunction',
'Age',
'Outcome']

df = pd.read_csv(
"pima-indians-diabetes.data.csv",
names=column_names
)

print("Dataset loaded successfully by Jithin_34")

# Data Preprocessing
columns_with_zero = [
'Glucose',
'BloodPressure',
'SkinThickness',
'Insulin',
'BMI'
]
for col in columns_with_zero:
 df[col] = df[col].replace(0, np.nan)
 df[col].fillna(df[col].median(), inplace=True)
 X = df.drop('Outcome', axis=1)
 y = df['Outcome']
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
# Logistic Regression without Scaling
model_no_scaling = LogisticRegression(max_iter=1000)
model_no_scaling.fit(X_train, y_train)
y_pred_no_scaling = model_no_scaling.predict(X_test)
# Evaluation without Scaling
acc_no = accuracy_score(y_test, y_pred_no_scaling)
prec_no = precision_score(y_test, y_pred_no_scaling)
rec_no = recall_score(y_test, y_pred_no_scaling)
f1_no = f1_score(y_test, y_pred_no_scaling)
```

```
# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Logistic Regression with Scaling
model_scaling = LogisticRegression(max_iter=1000)
model_scaling.fit(X_train_scaled, y_train)
y_pred_scaling = model_scaling.predict(X_test_scaled)
# Evaluation with Scaling
acc_scale = accuracy_score(y_test, y_pred_scaling)
prec_scale = precision_score(y_test, y_pred_scaling)
rec_scale = recall_score(y_test, y_pred_scaling)
f1_scale = f1_score(y_test, y_pred_scaling)
# Display Resultspima
print("WITHOUT FEATURE SCALING")
print("Accuracy :", acc_no)
print("Precision:", prec_no)
print("Recall :", rec_no)
print("F1 Score :", f1_no)
print("\nWITH FEATURE SCALING")
print("Accuracy :", acc_scale)
print("Precision:", prec_scale)
print("Recall :", rec_scale)
print("F1 Score :", f1_scale)
# Comparison Table
results = pd.DataFrame({
'Metric':['Accuracy',
'Precision',
'Recall',
'F1 Score'],'Without Scaling':[acc_no,
prec_no,
rec_no,
f1_no],
'With Scaling':[acc_scale,
prec_scale,
rec_scale,
f1_scale]
})
print("\nComparison of Models")
print(results)
```

```
===op

dataset Loaded successfully by Jithin_34.

WITHOUT FEATURE SCALING
Accuracy : 0.7532467532467533
Precision: 0.6666666666666666
Recall : 0.6181818181818182
F1 Score : 0.6415094339622641

WITH FEATURE SCALING
Accuracy : 0.7532467532467533
Precision: 0.6666666666666666
Recall : 0.6181818181818182
F1 Score : 0.6415094339622641

Comparison of Models
      Metric  Without Scaling  With Scaling
0   Accuracy         0.753247      0.753247
1  Precision         0.666667      0.666667
2     Recall         0.618182      0.618182
3   F1 Score         0.641509      0.641509
```

===e4

```
import pandas as pd
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
# Load Dataset
categories = [
    'comp.graphics',
```

```
    'rec.sport.baseball',
    'sci.space',
    'talk.politics.misc'
]
newsgroups = fetch_20newsgroups(
  subset='all',
  categories=categories,
  remove=('headers', 'footers', 'quotes')
)
print("Dataset loaded successfully by Jithin_34")
X_text = newsgroups.data
y = newsgroups.target
# Train-Test Split
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text,
    y,
    test_size=0.2,
    random_state=42
)
# Count Features for Multinomial NB
count_vectorizer = CountVectorizer(
    stop_words='english',
    max_features=5000
)
X_train_count = count_vectorizer.fit_transform(X_train_text)
X_test_count = count_vectorizer.transform(X_test_text)
# Binary Features for Bernoulli NB
binary_vectorizer = CountVectorizer(
    stop_words='english',
    binary=True,
    max_features=5000
)
X_train_binary = binary_vectorizer.fit_transform(X_train_text)
X_test_binary = binary_vectorizer.transform(X_test_text)
# Multinomial Naive Bayes
mnb = MultinomialNB()
mnb.fit(X_train_count, y_train)
y_pred_mnb = mnb.predict(X_test_count)
acc_mnb = accuracy_score(y_test, y_pred_mnb)
f1_mnb = f1_score(
    y_test,
    y_pred_mnb,
    average='weighted'
)
# Bernoulli Naive Bayes
bnb = BernoulliNB()
bnb.fit(X_train_binary, y_train)
y_pred_bnb = bnb.predict(X_test_binary)
acc_bnb = accuracy_score(y_test, y_pred_bnb)
f1_bnb = f1_score(
    y_test,
    y_pred_bnb,
    average='weighted'
)
# Results
print("Multinomial Naive Bayes")
print("Accuracy :", acc_mnb)
print("F1 Score :", f1_mnb)
print("\nBernoulli Naive Bayes")
print("Accuracy :", acc_bnb)
print("F1 Score :", f1_bnb)
# Comparison Table
results = pd.DataFrame({
    'Model': ['Multinomial NB',
    'Bernoulli NB'],
    'Accuracy': [acc_mnb,
    acc_bnb],
    'F1 Score': [f1_mnb,
    f1_bnb]
})
print("\nComparison")
print(results)
# Classification Reports
print("\nMultinomial NB Report")
print(classification_report(
    y_test,
    y_pred_mnb,
```

```
    target_names=newsgroups.target_names
))
print("\nBernoulli NB Report")
print(classification_report(
    y_test,
    y_pred_bnb,
    target_names=newsgroups.target_names
))
```

```
===op

dataset Loaded successfully by Jithin_34.

Multinomial Naive Bayes
Accuracy : 0.886058981233244
F1 Score : 0.8863087919544264

Bernoulli Naive Bayes
Accuracy : 0.774798927613941
F1 Score : 0.7774302422217592

Comparison
            Model  Accuracy  F1 Score
0  Multinomial NB  0.886059  0.886309
1    Bernoulli NB  0.774799  0.777430

Multinomial NB Report
                    precision    recall  f1-score   support

     comp.graphics       0.92      0.89      0.90       210
rec.sport.baseball       0.82      0.95      0.88       192
         sci.space       0.95      0.84      0.89       196
talk.politics.misc       0.87      0.87      0.87       148

          accuracy                           0.89       746
         macro avg       0.89      0.89      0.89       746
      weighted avg       0.89      0.89      0.89       746


Bernoulli NB Report
                    precision    recall  f1-score   support

     comp.graphics       0.92      0.81      0.86       210
rec.sport.baseball       0.58      0.98      0.73       192
         sci.space       0.92      0.69      0.79       196
talk.politics.misc       0.93      0.56      0.70       148

          accuracy                           0.77       746
         macro avg       0.84      0.76      0.77       746
      weighted avg       0.84      0.77      0.78       746
```

===e5

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

# Load Fashion MNIST Dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
print("Dataset loaded successfully by Jithin_34")

# Preprocessing

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten images
X_train = X_train.reshape(len(X_train), -1)
X_test = X_test.reshape(len(X_test), -1)
```

```
# Experiment with Different K Values
k_values = [1000, 3000, 5000, 7000, 9000]
results = []

for k in k_values:

    # Create KNN model
    knn = KNeighborsClassifier(n_neighbors=k)

    # Train model
    knn.fit(X_train, y_train)

    # Prediction
    y_pred = knn.predict(X_test)

    # Metrics
    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(
        y_test,
        y_pred,
        average='weighted'
    )

    recall = recall_score(
        y_test,
        y_pred,
        average='weighted'
    )

    f1 = f1_score(
        y_test,
        y_pred,
        average='weighted'
    )

    # Store results
    results.append(
        [k, accuracy, precision, recall, f1]
    )

    # Print results for each K
    print(f"\nK = {k}")
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1 Score :", f1)


# Results Table
results_df = pd.DataFrame(
    results,
    columns=[
        'K',
        'Accuracy',
        'Precision',
        'Recall',
        'F1 Score'
    ]
)

print("\nResults Summary")
print(results_df)


# Accuracy Plot
plt.figure(figsize=(8, 5))

plt.plot(
    results_df['K'],
    results_df['Accuracy'],
    marker='o'
)

plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy vs K")
```

```
plt.grid(True)
plt.show()
```

```
===op

dataset Loaded successfully by Jithin_34.

K = 1000
Accuracy : 0.7571
Precision: 0.7697892177221218
Recall   : 0.7571
F1 Score : 0.7578036920511965

K = 3000
Accuracy : 0.723
Precision: 0.7361034378273389
Recall   : 0.723
F1 Score : 0.7227780227628812

K = 5000
Accuracy : 0.6955
Precision: 0.7087464955841974
Recall   : 0.6955
F1 Score : 0.6935628559519182

K = 7000
Accuracy : 0.67
Precision: 0.6841434494625981
Recall   : 0.67
F1 Score : 0.6657475049502779

K = 9000
Accuracy : 0.6414
Precision: 0.6576833346626477
Recall   : 0.6414
F1 Score : 0.6331126102988058

Results Summary
      K  Accuracy  Precision  Recall  F1 Score
0  1000    0.7571   0.769789  0.7571  0.757804
1  3000    0.7230   0.736103  0.7230  0.722778
2  5000    0.6955   0.708746  0.6955  0.693563
3  7000    0.6700   0.684143  0.6700  0.665748
4  9000    0.6414   0.657683  0.6414  0.633113
```

===e6

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

print("Jithin Sunildas")

df = pd.read_excel("OnlineRetail.xlsx")
df = df.dropna()
df["Total Amount"] = df["Quantity"] * df["UnitPrice"]
customer_data = df.groupby("CustomerID").agg({
    "Quantity": "sum",
    "Total Amount": "sum",
    "InvoiceNo": "nunique"
}).reset_index()
customer_data.columns = ["CustomerID",
                         "TotalQuantity",
                         "TotalSpending",
                         "PurchaseFrequency"]
customer_data["Segment"] = pd.qcut(
    customer_data["TotalSpending"],
    q=3,
    labels=[
        "low",
        "medium",
        "High"]
)
```

```
X = customer_data[
    ["TotalQuantity",
     "TotalSpending",
     "PurchaseFrequency"]
]
y = customer_data["Segment"]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_})
print("\n feature Importance")
print(importance)
plt.figure(figsize=(16, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=encoder.classes_,
    filled=True)
plt.show()

===op

Jitin Sunildas
Accuracy: 0.9988571428571429

 feature Importance
             Feature  Importance
0      TotalQuantity         0.0
1      TotalSpending         1.0
2  PurchaseFrequency         0.0

===e7

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

df = pd.read_csv("adult.csv", header=None, skiprows=1)

df.columns = [
    "age",
    "workclass",
    "fnlwgt",
    "education",
    "education_num",
    "marital_status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "capital_gain",
    "capital_loss",
    "hours_per_week",
    "native_country",
```

```
    "income"
]

print("Dataset loaded successfully by Jithin_34")

df.replace("?", np.nan, inplace=True)
df.dropna(inplace=True)

label_encoders = {}

for column in df.columns:
    if not pd.api.types.is_numeric_dtype(df[column]):
        le = LabelEncoder()
        df[column] = le.fit_transform(df[column])
        label_encoders[column] = le

print(df.columns)
X = df.drop("income", axis=1)
y = df["income"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

def evaluate(y_true, y_pred, model_name):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    print("\n", model_name)
    print("*" * 30)
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1 Score :", f1)

evaluate(y_test, lr_pred, "Logistic Regression")
evaluate(y_test, dt_pred, "Decision Tree")

print("\nClassification Report - Logistic Regression")
print(classification_report(y_test, lr_pred))

print("\nClassification Report - Decision Tree")
print(classification_report(y_test, dt_pred))

plt.figure(figsize=(18, 10))

plot_tree(
    dt_model,
    feature_names=X.columns,
    filled=True,
    fontsize=8
)

plt.show()

===op

dataset Loaded successfully by Jithin_34.

Columns: ['age', 'workclass', 'fnlwgt', 'education', 'education_num', 'marital_status', 'occupation', 'relationship',
'race', 'sex', 'capital_gain', 'capital_loss', 'hours_per_week', 'native_country', 'income']

 Logistic Regression
******************************
```

```
Accuracy : 0.8036484245439469
Precision: 0.6813933729821581
Recall   : 0.3640490240581026
F1 Score : 0.47455621301775147

 Decision Tree
******************************
Accuracy : 0.8064123825317855
Precision: 0.600088573959256
Recall   : 0.6150703586019065
F1 Score : 0.6074871105133378

Classification Report - Logistic Regression
              precision    recall  f1-score   support

           0       0.82      0.95      0.88      6842
           1       0.68      0.36      0.47      2203

    accuracy                           0.80      9045
   macro avg       0.75      0.65      0.68      9045
weighted avg       0.79      0.80      0.78      9045


Classification Report - Decision Tree
              precision    recall  f1-score   support

           0       0.88      0.87      0.87      6842
           1       0.60      0.62      0.61      2203

    accuracy                           0.81      9045
   macro avg       0.74      0.74      0.74      9045
weighted avg       0.81      0.81      0.81      9045
```

===e8

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
# Load Iris Dataset
iris = datasets.load_iris()
print("Dataset loaded successfully by Jithin_34")
# Use only Petal Length and Petal Width
X = iris.data[:, [2, 3]]
# Binary Classification
# Setosa = 1
# Non-Setosa = 0
y = (iris.target == 0).astype(int)
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.3,
random_state=42
)
# Linear SVM
svm_model = SVC(
kernel='linear',
C=1
)
svm_model.fit(X_train, y_train)
# Prediction
y_pred = svm_model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Plot Decision Boundary
plt.figure(figsize=(8,6))
plt.scatter(
X[:,0],
X[:,1],
c=y,
```

```
cmap='coolwarm'
)
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = svm_model.decision_function(xy)
Z = Z.reshape(XX.shape)
# Decision Boundary
ax.contour(
XX,
YY,
Z,
colors='k',
levels=[-1, 0, 1],
alpha=0.8,
linestyles=['--','-','--']
)
# Support Vectors
ax.scatter(
svm_model.support_vectors_[:,0],
svm_model.support_vectors_[:,1],
s=100,
linewidth=1,
facecolors='none',
edgecolors='black'
)
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("Linear SVM Decision Boundary and Margin")
plt.show()
```

```
===op

dataset Loaded successfully by Jithin_34.

Accuracy: 1.0

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        26
           1       1.00      1.00      1.00        19

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

===e9

```
import numpy as np
import pandas as pd
from tensorflow.keras.datasets import fashion_mnist
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
# Load Fashion MNIST
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
print("Dataset loaded successfully by Jithin_34")
# Normalize Images
X_train = X_train / 255.0
X_test = X_test / 255.0
# Flatten Images
X_train = X_train.reshape(len(X_train), -1)
X_test = X_test.reshape(len(X_test), -1)
# To reduce training time
X_train = X_train[:10000]
y_train = y_train[:10000]
X_test = X_test[:2000]
y_test = y_test[:2000]

# Linear Kernel
```

```
linear_svm = SVC(
kernel='linear'
)
linear_svm.fit(X_train, y_train)
linear_pred = linear_svm.predict(X_test)
# Polynomial Kernel
poly_svm = SVC(
kernel='poly',
degree=3
)
poly_svm.fit(X_train, y_train)
poly_pred = poly_svm.predict(X_test)
# RBF Kernel
rbf_svm = SVC(
kernel='rbf'
)
rbf_svm.fit(X_train, y_train)
rbf_pred = rbf_svm.predict(X_test)
# Evaluation Function
def evaluate_model(y_true, y_pred, model_name):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(
        y_true,
        y_pred,
        average='weighted'
    )
    recall = recall_score(
        y_true,
        y_pred,
        average='weighted'
    )
    f1 = f1_score(
        y_true,
        y_pred,
        average='weighted'
    )
    print("\n", model_name)
    print("-"*40)
    print("Accuracy :", accuracy)
    print("Precision:", precision)

    print("Recall :", recall)
    print("F1 Score :", f1)

# Results
evaluate_model(y_test, linear_pred, "Linear SVM")
evaluate_model(y_test, poly_pred, "Polynomial SVM")
evaluate_model(y_test, rbf_pred, "RBF SVM")
```

===op

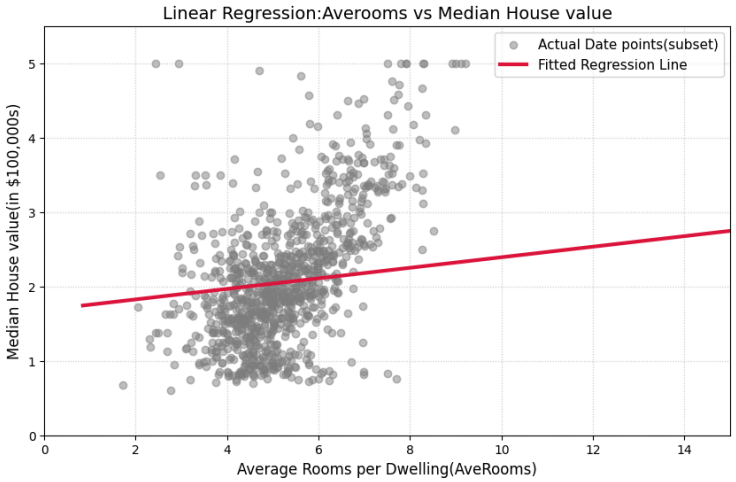
```
dataset Loaded successfully by Jithin_34.

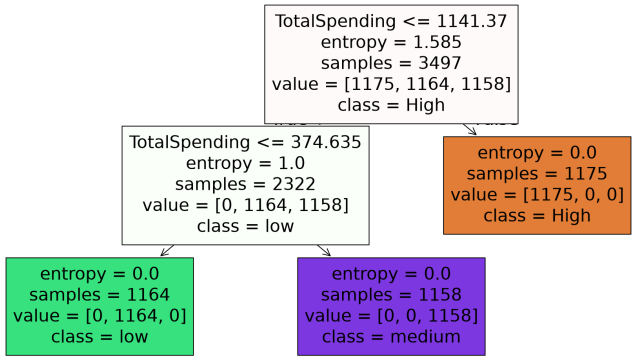
 Linear SVM
----------------------------------------
Accuracy : 0.8305
Precision: 0.8333224573650666
Recall : 0.8305
F1 Score : 0.8312263902125148

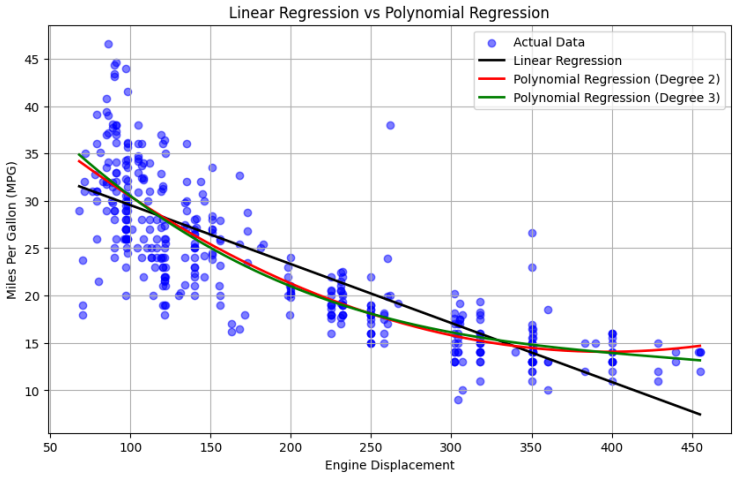
 Polynomial SVM
----------------------------------------
Accuracy : 0.828
Precision: 0.840998785391224
Recall : 0.828
F1 Score : 0.8308992029690123

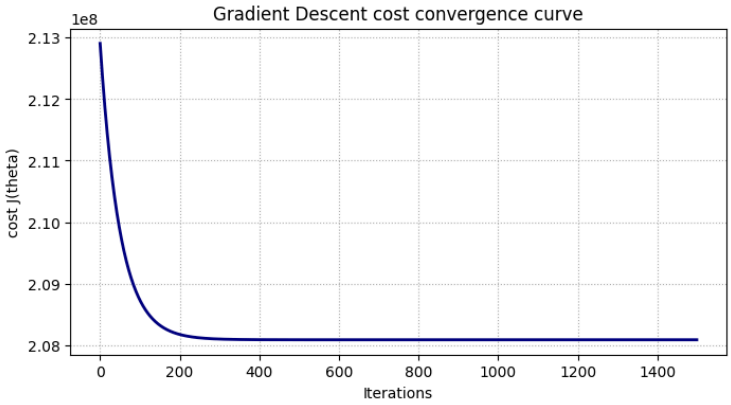
 RBF SVM
----------------------------------------
Accuracy : 0.86
Precision: 0.8616455332799008
Recall : 0.86
F1 Score : 0.8603642500236333
```

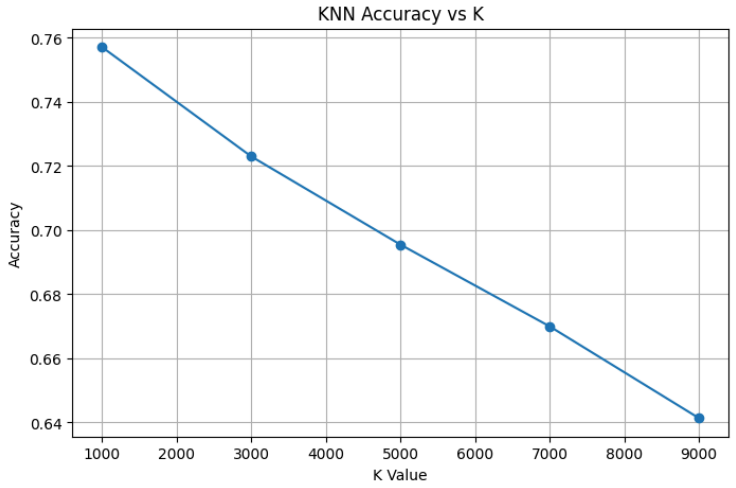
===e10

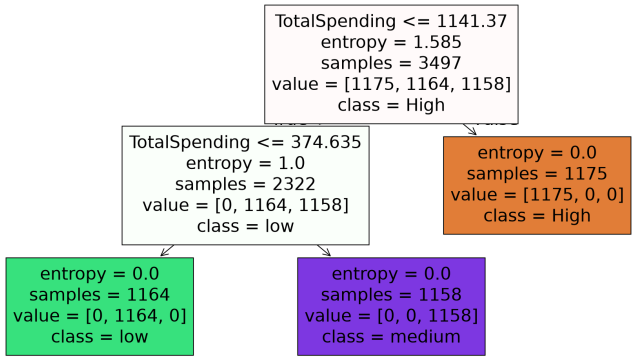


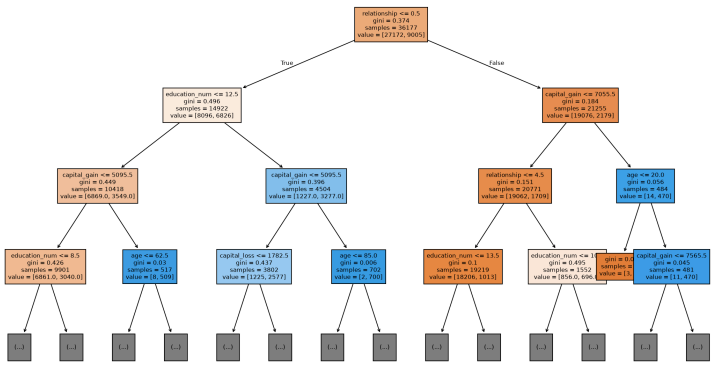












```
#image("e1f1.png")
#image("e2f1.png")
#image("e6f1.png")
#image("e7f1.png")
#image("e1f2.png")
#image("e7f1.png")
#image("e8f1.png")
```